

SUMMARY

This topic covers Hashing in Data Structures, detailing its mechanics, collision resolution techniques, and practical challenges for achieving efficient data retrieval.

Hashing aims to achieve constant-time search performance ($O(1)$), surpassing the logarithmic search complexity of balanced trees like AVL or B-Trees. By using a hash function to map keys directly to table indices, hashing allows near-instant access to data. Collisions—when multiple keys map to the same index—pose challenges and are resolved using methods such as Linear Probing, Quadratic Probing, Double Hashing, and Separate Chaining. Practical considerations include clustering, deletion issues, and load factor management, all of which affect overall efficiency.

1. THE EVOLUTION OF SEARCH EFFICIENCY

Data Structure	Search Complexity (Average/Worst Case)	Basis of Performance
Unsorted Array	$O(n)$	Linear search through all elements
Binary Search Tree (BST)	$O(n)$ (Worst Case)	Dependent on height; can become skewed
AVL Tree	$O(\log n)$	Height-balanced to ensure logarithmic search
B-Tree / B+ Tree	$O(\log_m n)$	Increased width reduces disk access (commonly used in DBMS)
Hashing	$O(1)$ (Average Case)	Direct mapping via mathematical function

The goal of hashing is to achieve $O(1)$ search time, independent of the number of elements.

2. FUNDAMENTAL MECHANICS OF HASHING

2.1 Hash Functions and Mapping

- Hashing maps a key to a table index using a hash function: $y = f(\text{key})$
- Example: $h(k) = k \bmod m$, where m is the table size

2.2 Characteristics of a Good Hash Function

1. Ease of Computation: Must be fast to compute.
2. Uniform Distribution: Keys should be evenly spread to prevent clustering.
3. Minimal Collisions: Perfect hashing is rare, but a good function minimizes collisions.

3. COLLISION RESOLUTION TECHNIQUES (OPEN ADDRESSING)

Collisions occur when multiple keys map to the same index. Open addressing searches for alternative empty slots.

3.1 Linear Probing

- Function: $h(k, i) = (h(k) + i) \bmod m$
- Primary Clustering: Sequential filled slots form clusters, slowing insertions and searches

3.2 Quadratic Probing

- Function: $h(k, i) = (h(k) + i^2) \bmod m$
- Avoids primary clustering but suffers secondary clustering: keys with same initial hash follow same resolution path
- May fail to find a free slot even if table is partially empty

3.3 Double Hashing

- Function: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$
- Secondary hash function $h_2(k)$ must never return zero
- Minimizes clustering but increases computation time

4. SEPARATE CHAINING

- Collisions resolved using a linked list at each table index
- Load Factor $\alpha = n / m$
 - In Open Addressing, $\alpha \leq 1$
 - In Chaining, α can exceed 1
- Performance:
 - Insertion: $O(1)$ (insert at list head)
 - Search/Deletion: Worst-case $O(n)$ if all keys collide, average case $O(1)$

5. PRACTICAL CHALLENGES IN HASHING

5.1 Deletion Problem

- In Open Addressing, deleting a key may break probe paths for other keys
- Solution: Use "deleted" markers or rehash remaining keys
- Separate Chaining avoids this issue since removing a node from a list does not break access to other nodes

5.2 Load Factor and Table Capacity

- Open Addressing: Performance degrades rapidly as $\alpha \rightarrow 1$
- Separate Chaining: Can handle higher load factors but requires additional memory for linked lists.

Hashing offers highly efficient average-case retrieval ($O(1)$), but effectiveness depends on:

- Choice of hash function
- Collision resolution strategy
- Proper management of deletion and load factor

Open Addressing minimizes clustering but complicates deletions, whereas Separate Chaining handles high loads and deletion more flexibly but at a potential memory cost.

